Creating and Spring-Managing Beans Using Java-Based Configuration (Java Type)

Project: spring-managed-beans-using-java-type

Step 1: Create the Maven project in Eclipse

- File → New → Maven Project → set the workspace location path → Next
- Set Catalog to Internal and select the archetype → Next
- Set Group Id (`com.amazon.user`), Artifact Id (`spring-managed-beans-using-java-type`), Version (`0.0.1-SNAPSHOT`) and Package (`com.amazon.user`) → Finish
- Type Y in the Console when prompted

Step 2: Add the Spring Context dependency

A project is created in Eclipse's Package Explorer with the name of the artifact we set, `spring-managed-beans-using-java-type`.

Project structure in Package Explorer

Open `pom.xml` and add the `spring-context` dependency from https://mvnrepository.com/artifact/org.springframework/spring-context. Always add one version below the latest.

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.19</version>
    <scope>compile</scope>
</dependency>
```

Note: Java 17 is used in this project.

Step 3: Create the POJO class

Create the POJO class `Employee` in package `com.amazon.employee`.

Employee.java

```
package com.amazon.employee;

public class Employee {

    private String name;
    private String id;
    private String band;

    public Employee(String name, String id, String band) {
        super();
        this.name = name;
        this.id = id;
        this.band = band;
    }

    public void displayInfo() {
        System.out.println("name=" + name + ", id=" + id + ", band=" + band);
    }
}
```

Step 4: Create the Spring configuration class

Create the class `SpringConfig` in package `com.amazon.config`. Annotate it with `@Configuration`, and create the beans using `@Bean` methods (`emp1`, `emp2` and `createEmp3`).

SpringConfig.java

```
package com.amazon.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import com.amazon.employee.Employee;

@Configuration
public class SpringConfig {

    @Bean("emp1")
    public Employee createEmp1() {
        Employee employee = new Employee("Alpha", "EMP-101", "PREMIUM");
        return employee;
    }

    @Bean("emp2")
    public Employee createEmp2() {
        Employee employee = new Employee("Beta", "EMP-102", "SEMI-PREMIUM");
        return employee;
    }

    // if no bean name is written then method name will act like bean id
    @Bean
    public Employee createEmp3() {
        Employee employee = new Employee("Charlie", "EMP-103", "REGULAR");
        return employee;
    }
}
```

Key Points

@Configuration
It tells Spring that this class is the source of bean definitions. Spring reads this class to know which beans it has to create and manage.

@Bean
It tells the actual bean definition. The method with `@Bean` creates an object, and the object it returns is registered in the Spring container as a bean. If we write a name inside `@Bean`, that name is the bean id. If we don't write any name, the method name acts as the bean id.

Step 5: Load the beans in App.java

In `App.java` (package `com.amazon.user`), create the Spring container using `AnnotationConfigApplicationContext` and pass `SpringConfig.class` to it. Then get the beans `emp1`, `emp2` and `createEmp3` from the container and call `displayInfo()` on each.

App.java

```
package com.amazon.user;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import com.amazon.config.SpringConfig;
import com.amazon.employee.Employee;

public class App {
    public static void main(String[] args) {
        System.out.println("Creating Spring Beans using Java...");

        ApplicationContext context = new AnnotationConfigApplicationContext(SpringConfig.class);

        Employee emp1 = context.getBean("emp1", Employee.class);
        emp1.displayInfo();

        Employee emp2 = context.getBean("emp2", Employee.class);
        emp2.displayInfo();

        Employee emp3 = context.getBean("createEmp3", Employee.class);
        emp3.displayInfo();
    }
}
```

Step 6: Run the application and check the output

Run `App.java` as a Java Application. The console prints all three Employee beans created by Spring.

Output

```
Creating Spring Beans using Java...
name=Alpha, id=EMP-101, band=PREMIUM
name=Beta, id=EMP-102, band=SEMI-PREMIUM
name=Charlie, id=EMP-103, band=REGULAR
```

Note: If we write a wrong bean id or a wrong method name, we immediately get `NoSuchBeanDefinitionException`.